Intermediate (build the loop yourself) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `react_loop_practice.py` (Intermediate section): an agent is not a new idea — it's Doc06's tool round trip, repeated until the model stops asking for tools. Building the loop yourself, once, is what makes `create_agent` and Project 2's Worker readable instead of magic. **If not:** the Build Task's `agent.py` would be the first loop you ever wrote, with no smaller version to trust.

Copy Doc06's Build Task files into `practice/`, unchanged: `tools.py`, `http_client.py`, `exceptions.py`, `logging_setup.py`. Run everything from inside `practice/`.

Basic Version

Approach 1 — one tool, one loop

```
# react_loop_practice.py — Intermediate section
from langchain_openai import ChatOpenAI

from tools import add

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
model_with_tools = model.bind_tools([add])

messages = [{"role": "user", "content": "Add 1 + 2, then add 3 to that."}]
for step in range(5):
    response = model_with_tools.invoke(messages)
    messages.append(response)
    if not response.tool_calls:
        print("Final answer:", response.content)
        break
    for call in response.tool_calls:
        result = add.invoke(call["args"])
        print(f"step {step}: add({call['args']}) -> {result}")
        messages.append(
            {"role": "tool", "content": str(result), "tool_call_id": call["id"]}
        )
```

Expected output (model wording varies):

```
step 0: add({'a': 1, 'b': 2}) -> 3
step 1: add({'a': 3, 'b': 3}) -> 6
Final answer: 1 + 2 is 3, and adding 3 gives 6.
```

It works, and the loop can never run forever (`range(5)`). But nothing is returned to a caller, nothing records the steps, and running out of steps just ends silently.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — `run_agent()`, a step log, and a named error

Story — `run_agent()`: the loop becomes a function a caller can use: it returns an `AgentResult` (the answer plus every step), and running out of steps raises `MaxIterationsExceeded` with the partial log. **If not:** a run that never finished would look the same as one that finished with an empty answer.

```
# react_loop_practice.py — Intermediate section
from dataclasses import dataclass, field

from langchain_openai import ChatOpenAI

from tools import add


class MaxIterationsExceeded(Exception):
    def __init__(self, message: str, steps: list[dict]) -> None:
        # why: the caller needs the partial log to see why it never
        # finished — the same "error with details" idea as Doc02's
        # PermanentHTTPError
        self.steps = steps
        super().__init__(message)


@dataclass
class AgentResult:
    final_answer: str
    # why: a list default needs field(default_factory=list), not `= []` —
    # otherwise every AgentResult would share one list
    steps: list[dict] = field(default_factory=list)


def run_agent(task: str, max_iterations: int = 5) -> AgentResult:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools([add])
    messages = [{"role": "user", "content": task}]
    steps: list[dict] = []

    for step in range(max_iterations):
```

```
        # THINK: the model answers now, or asks for a tool
        response = model_with_tools.invoke(messages)
        messages.append(response)
        if not response.tool_calls:
            # when: no tool call means the model is done — the only exit
            return AgentResult(final_answer=response.content, steps=steps)

        for call in response.tool_calls:
            # ACT, then OBSERVE: run the tool, send the result back
            result = str(add.invoke(call["args"]))
            messages.append(
                {"role": "tool", "content": result, "tool_call_id": call["id"]}
            )
            steps.append({"step": step, "tool": call["name"], "result": result})

    message = f"No answer after {max_iterations} steps"
    raise MaxIterationsExceeded(message, steps)


if __name__ == "__main__":
    result = run_agent("Add 1 + 2, then add 3 to that.")
    print(f"Final answer: {result.final_answer}")
    for step in result.steps:
        print(step)
```

Expected output (model wording varies):

```
Final answer: 1 + 2 is 3, and adding 3 gives 6.
{'step': 0, 'tool': 'add', 'result': '3'}
{'step': 1, 'tool': 'add', 'result': '6'}
```

> Back to the exercise · Hint 1 · Hint 2 · Solution

Approach 2 — every Doc06 tool, failures as Observations, and a logger

Story — `run_tool()` and the full loop: with more than one tool, the loop must find the tool by name (Doc06's `find_tool()`), and a failing tool must become an `"Error: ..."` Observation the model can react to — never a crash. Every step is logged with Doc01's `get_logger()`. **If not:** one bad argument or one flaky tool would end the whole run, and the only record of it would be a traceback.

```
# react_loop_practice.py — Intermediate section
from dataclasses import dataclass, field

from langchain_core.tools import BaseTool
from langchain_openai import ChatOpenAI

from logging_setup import get_logger
from tools import ALL_TOOLS


class MaxIterationsExceeded(Exception):
    def __init__(self, message: str, steps: list[dict]) -> None:
        self.steps = steps
        super().__init__(message)


@dataclass
class AgentResult:
    final_answer: str
    steps: list[dict] = field(default_factory=list)


def find_tool(tools: list[BaseTool], name: str) -> BaseTool | None:
    # how: Doc06's Build Task function, copied unchanged
    for candidate in tools:
        if candidate.name == name:
            return candidate
    return None


def run_tool(
    call: dict, tools: list[BaseTool]
) -> tuple[str | None, str | None]:
    # how: returns (result, None) on success, (None, error) on failure
```

```
    tool = find_tool(tools, call["name"])
    if tool is None:
        return None, f"unknown tool: {call['name']}"
    try:
        # how: .invoke() checks the arguments with the tool's Pydantic
        # args_schema first (Doc06) — bad arguments never reach the tool
        return str(tool.invoke(call["args"])), None
    except Exception as e:
        # why: any tool failure becomes an Observation, never a crash
        return None, str(e)


def run_agent(
    task: str, max_iterations: int = 5, tools: list[BaseTool] | None = None
) -> AgentResult:
    # why: None, not a list, as the default — a list default would be one
    # shared list; None means "use ALL_TOOLS", and a test can pass its own
    if tools is None:
        tools = ALL_TOOLS
    logger = get_logger(__name__)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools(tools)
    messages = [{"role": "user", "content": task}]
    steps: list[dict] = []

    for step in range(max_iterations):
        response = model_with_tools.invoke(messages)
        messages.append(response)
        if not response.tool_calls:
            return AgentResult(final_answer=response.content, steps=steps)

        # why: run EVERY call the model asked for — each needs an answer
        for call in response.tool_calls:
            result, error = run_tool(call, tools)
            if error is None:
                observation = result
                logger.info(f"step {step}: {call['name']} -> {result}")
            else:
                observation = f"Error: {error}"
                # how: a warning, not an error — the loop keeps going
                logger.warning(f"step {step}: {call['name']} failed: {error}")
            messages.append({
                "role": "tool",
                "content": observation,
                "tool_call_id": call["id"],
            })
            steps.append({
```

```
                "step": step,
                "tool": call["name"],
                "arguments": call["args"],
                "result": result,
                "error": error,
            })

    message = f"No answer after {max_iterations} steps"
    raise MaxIterationsExceeded(message, steps)


if __name__ == "__main__":
    result = run_agent("What's 5 + 7?")
    print(f"Final answer: {result.final_answer}")
    for step in result.steps:
        print(step)
```

Expected output (the log lines go to the terminal too, unless `LOG_LEVEL=WARNING`):

```
Final answer: 5 + 7 is 12.
{'step': 0, 'tool': 'add', 'arguments': {'a': 5, 'b': 7}, 'result': '12',
 'error': None}
```

(The step dict is shown wrapped onto 2 lines to fit the page — really one line.)

Difference from Approach 1: `run_tool()` returns `(result, error)`, so an unknown tool, bad arguments (rejected by the tool's Pydantic `args_schema` — Doc06) or a crashing tool all become an Observation. Every call the model asks for gets an answer, and every step is logged at `info` or `warning`. The optional `tools` parameter (default `None`, then `ALL_TOOLS`) lets a test pass its own tools.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Approach 3 — a factory that builds a fresh fail-once tool

Story — `make_flaky_add()`: to prove the loop recovers, you need a tool that fails on purpose — once — and a *new* one for every test, so tests never share a counter. A function that builds and returns a tool does exactly that. **If not:** a failure could only be tested by luck, and one test's failure count would leak into the next.

```
# react_loop_practice.py — add below run_agent()
from langchain_core.tools import tool


class FlakyState:
    def __init__(self) -> None:
        self.call_count = 0


def make_flaky_add(state: FlakyState) -> BaseTool:
    # how: a function that builds and returns a NEW tool; the tool inside
    # keeps using the `state` it was built with — so every tool made here
    # has its own counter, and two tests never share one
    @tool
    def flaky_add(a: int, b: int) -> int:
        """Add two integers together and return the sum."""
        state.call_count = state.call_count + 1
        if state.call_count == 1:
            # why: fails on its first call only, on purpose
            raise ConnectionError("temporarily unavailable")
        return a + b

    return flaky_add


def check_recovery() -> None:
    call = {"name": "flaky_add", "args": {"a": 2, "b": 3}}
    flaky_add = make_flaky_add(FlakyState())
    print(f"case 1 (first call)   -> {run_tool(call, [flaky_add])}")
    print(f"case 2 (second call)  -> {run_tool(call, [flaky_add])}")
    fresh_add = make_flaky_add(FlakyState())
    print(f"case 3 (fresh tool)   -> {run_tool(call, [fresh_add])}")

    # how: the whole loop, with a fresh fail-once tool passed in
    result = run_agent("What's 2 + 3?", 5, [make_flaky_add(FlakyState())])
```

```
    errors = 0
    for step in result.steps:
        if step["error"] is not None:
            errors = errors + 1
    print(f"case 4 (whole loop)   -> {errors} failed step, then: "
          f"{result.final_answer}")
```

Expected output (call `check_recovery()` at the bottom of the file):

```
case 1 (first call)   -> (None, 'temporarily unavailable')
case 2 (second call)  -> ('5', None)
case 3 (fresh tool)   -> (None, 'temporarily unavailable')
case 4 (whole loop)   -> 1 failed step, then: 2 + 3 is 5.
```

> Back to the exercise · Hint 1 · Hint 2 · Solution

Approach 4 — check the arguments yourself, before the tool runs

Story — `run_tool_checked()`: a tool's `args_schema` checks arguments inside `.invoke()`. Checking them yourself first, with the tool's own Pydantic model, gives your own clear message, and `model_dump()` turns the checked object back into the dict `.invoke()` needs. **If not:** the Build Task's rule "arguments must pass Pydantic checking *before* the tool runs" would be met only by accident.

```
# react_loop_practice.py — add below run_agent()
from pydantic import ValidationError

from tools import AddArgs

# how: tool name -> the Pydantic model its arguments must fit
ARG_MODELS = {"add": AddArgs}


def run_tool_checked(
    call: dict, tools: list[BaseTool]
) -> tuple[str | None, str | None]:
    tool = find_tool(tools, call["name"])
    if tool is None:
        return None, f"unknown tool: {call['name']}"
    try:
        if call["name"] in ARG_MODELS:
            # why: check the arguments yourself, BEFORE the tool runs,
            # with your own clear message when they're wrong
            checked = ARG_MODELS[call["name"]](**call["args"])
            # how: model_dump() turns the checked object back into a dict
            result = tool.invoke(checked.model_dump())
        else:
            result = tool.invoke(call["args"])
        return str(result), None
    except ValidationError:
        return None, f"invalid arguments for {call['name']}: {call['args']}"
    except Exception as e:
        return None, str(e)


def check_arguments() -> None:
    good = {"name": "add", "args": {"a": 2, "b": 3}}
    bad = {"name": "add", "args": {"a": "two", "b": 3}}
```

```
    print(f"case 1 (good args) -> {run_tool_checked(good, ALL_TOOLS)}")
    print(f"case 2 (bad args) -> {run_tool_checked(bad, ALL_TOOLS)}")
```

Expected output (call `check_arguments()` at the bottom of the file):

```
case 1 (good args) -> ('5', None)
case 2 (bad args) -> (None, "invalid arguments for add: {'a': 'two', 'b': 3}")
```

> Back to the exercise · Hint 1 · Hint 2 · Solution

Approach 5 — the same loop on the raw OpenAI SDK

Story — `react_loop_raw_sdk.py`: `bind_tools()` hides two jobs — describing each tool as a JSON schema, and turning the model's JSON arguments back into Python. Doing both by hand once, with Doc04's client, shows exactly what LangChain does for you. **If not:** tool calling would stay a LangChain feature in your head, not an API feature.

```
# react_loop_raw_sdk.py
import json

from openai import OpenAI

from tools import add

client = OpenAI()

# how: the raw SDK needs the tool described as a JSON schema by hand —
# the exact thing @tool and bind_tools() write for you
ADD_SCHEMA = {
    "type": "function",
    "function": {
        "name": "add",
        "description": "Add two integers together and return the sum.",
        "parameters": {
            "type": "object",
            "properties": {
                "a": {"type": "integer"},
                "b": {"type": "integer"},
            },
            "required": ["a", "b"],
        },
    },
}
# how: a dispatch table — tool name -> the tool that runs it
TOOLS_BY_NAME = {"add": add}


def run_agent_raw(task: str, max_iterations: int = 5) -> str:
    messages = [{"role": "user", "content": task}]
    for step in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini", messages=messages, tools=[ADD_SCHEMA]
```

```
        )
        message = response.choices[0].message
        messages.append(message)
        if not message.tool_calls:
            return message.content
        for call in message.tool_calls:
            # how: the raw SDK hands the arguments over as a JSON string
            args = json.loads(call.function.arguments)
            tool = TOOLS_BY_NAME[call.function.name]
            result = str(tool.invoke(args))
            print(f"step {step}: {call.function.name}({args}) -> {result}")
            messages.append(
                {"role": "tool", "tool_call_id": call.id, "content": result}
            )
    return "No answer — the step limit was reached."


if __name__ == "__main__":
    print(run_agent_raw("Add 1 + 2, then add 3 to that."))
```

Expected output (wording varies):

```
step 0: add({'a': 1, 'b': 2}) -> 3
step 1: add({'a': 3, 'b': 3}) -> 6
1 + 2 is 3, and adding 3 gives 6.
```

Difference from Approach 2: the schema is written by hand, the arguments arrive as a JSON string (`json.loads`), the id is `call.id` instead of `call["id"]` , and a `TOOLS_BY_NAME` dict does `find_tool()` 's job.

Which one should you actually write? Approach 2 — the next exercises import its `run_agent()` , and the Build Task's `agent.py` is this same file with one more tool. Approaches 3 and 4 are the two pieces the Build Task's Approaches 2 and 3 reuse; Approach 5 is how the loop looks without LangChain.